Abstract: Data-Mart-Erstellung in SQL (DLBDSPBDM01_D), Buchtausch-App

Ausgangslage und Zielsetzung

Ziel des Projekts war der Entwurf und die Umsetzung einer normalisierten relationalen Datenbank für eine Buchtausch-App, über die Nachbarinnen und Nachbarn Bücher zeitlich befristet verleihen und ausleihen. Die Datenbank muss bibliografische Daten, Personen- und Adressdaten, Verleihbedingungen, Übergabeorte mit Geokoordinaten, Zeitfenster, Ausleihvorgänge und Bewertungen aufnehmen und neben tabellarischen auch räumliche Abfragen ("Bücher in meiner Nähe") ermöglichen.

Konzept und Methodik

Die Arbeit folgte den drei Phasen des Portfolios. In der Konzeptionsphase entstanden eine Anforderungsspezifikation, ein ER-Modell in Chen-Notation mit elf Entitäten und zwei echten Dreifachbeziehungen sowie ein Datenwörterbuch. Zentrale Entwurfsentscheidung ist die Trennung von Buch (bibliografische Ausgabe) und Exemplar (konkrete Kopie einer Person), sodass Titel-, Autor- und Verlagsdaten nicht je Kopie wiederholt werden (dritte Normalform). Als DBMS wurde PostgreSQL gewählt, da die Aufgabenstellung kein System vorgibt und PostgreSQL deklarative Constraints, partielle Indizes und Trigger in PL/pgSQL bietet.

Umsetzung

In der Erarbeitungsphase wurde das Modell in ein Schema mit 14 Tabellen überführt. Die beiden Dreifachbeziehungen wurden als eigene Tabellen mit Surrogatschlüssel umgesetzt, weil eine Bewertung auf einen konkreten Ausleihvorgang verweisen muss. Die Testdaten erzeugt ein Python-Skript, das die Geschäftsregeln bereits beim Erzeugen prüft. Testfälle laufen automatisiert gegen die Datenbank; Negativtests werden in einer Transaktion mit ROLLBACK ausgeführt. Die Rückmeldungen zu beiden Zwischenabgaben führten zu gezielten Verbesserungen: Bewertungen sind an Ausleihvorgänge gekoppelt, Adressen stehen in einer 1:N-Beziehung zum Benutzer, der zeitabhängige CHECK auf das Erscheinungsjahr wurde durch feste Grenzen und einen Trigger ersetzt, ein partieller UNIQUE-Index verhindert doppelte offene Ausleihen, und weitere Trigger sichern die maximale Leihdauer sowie die Zuordnung von Zeitfenstern zu Besitzer und Angebot. Die räumliche Suche nutzt die Haversine-Formel direkt in SQL.

Ergebnis und Reflexion

Das Ergebnis umfasst 14 Tabellen mit 342 Datensätzen (jede Tabelle mit mindestens zehn Einträgen), 14 Primär-, 17 Fremdschlüssel- und 16 CHECK-Constraints, 8 Trigger sowie 37 automatisierte Testfälle (16 positive, 21 negative), die alle die erwartete Wirkung zeigen. Die Anforderungen der Aufgabenstellung sind damit erfüllt. Als wesentliche Erkenntnis zeigte sich, dass tabellenübergreifende Regeln (Zeitfenster gehört zum Besitzer, Bewertung nur nach Rückgabe) nicht als einfache CHECK-Constraints formulierbar sind, sondern Trigger erfordern, die wieder-

um durch Negativtests abgesichert werden müssen. Einschränkungen sind der Verzicht auf die Erweiterung PostGIS, die rein synthetischen Testdaten und fehlende Tests unter paralleler Last. Eingesetzt wurden PostgreSQL 16 (psql), Python 3 für Datengenerierung, Testautomatisierung und Dokumenterstellung sowie Git und GitHub.